

{creative} [coding]

Dokumentation Woche 5

{Montag} 25.05.2026

Pfingstmontag

{Dienstag} 26.05.2026

Vormittag

- Experimente mit Flock und Typografie
- Ausarbeitung Konzept

Nachmittag

- Feedbackgespräch mit Stefanie und Yann
- Weiters Vorgehen besprechen

★ FLOCK – TYPOGRAFIE SPUREN

```
// — Wort-Schwarm —————
// Wörter in setup() eintragen oder live per addWord("...") hinzufügen
// Maus ziehen = zufälliges Wort aus dem Pool spawnen

let flock;
let wordPool = []; // { word, brightness }
let started = false;

// Graustufen: erstes Wort = hell (220), letztes = dunkel (60)
function brightnessForIndex(i, total) {
  if (total <= 1) return 200;
  return Math.round(220 - (i / (total - 1)) * 160);
}

function addWord(val) {
  if (!val || wordPool.find(w => w.word === val)) return;

  wordPool.push({ word: val, brightness: 200 });

  // Helligkeiten für alle Wörter neu verteilen
  wordPool.forEach((e, i) => {
    e.brightness = brightnessForIndex(i, wordPool.length);
  });
}
```

NICHT REGISTRIERT ×

```

// Einen neuen Boid in der Bildschirmmitte spawnen
let entry = wordPool.find(w => w.word === val);
let b = new Boid(
  width/2 + random(-100, 100),
  height/2 + random(-100, 100),
  val,
  entry.brightness
);
flock.add(b);

// Helligkeiten aller bestehenden Boids aktualisieren
flock.boids.forEach(b => {
  let e = wordPool.find(w => w.word === b.word);
  if (e) b.brightness = e.brightness;
});

started = true;
}

// — p5.js Einstiegspunkte —————

function setup() {
  createCanvas(windowWidth, windowHeight);
  textFont('monospace');
  flock = new Flock();

  // — Wörter hier eintragen —————
  addWord("Drum & Bass");
  addWord("Windrush-Generation");
  addWord("Carribean");
  addWord("Great Britan");
  addWord("Jungle");
  addWord("D'n'B");
}

function draw() {
  //background(10);

  if (!started) {
    fill(255, 255, 255, 45);
    noStroke();
    textSize(13);
    textAlign(CENTER, CENTER);
    text('addWord("dein Wort") aufrufen um zu starten', width/2, height/2);
    return;
  }

  flock.run();
}

// Maus ziehen: zufälliges Wort aus dem Pool an Mausposition spawnen
function mouseDragged() {
  if (!started || wordPool.length === 0) return;
  let entry = wordPool[floor(random(wordPool.length))];
  flock.add(new Boid(mouseX, mouseY, entry.word, entry.brightness));
}

function windowResized() {

```

```

    resizeCanvas(windowWidth, windowHeight);
}

// — Flock —————

class Flock {
  constructor() { this.boids = []; }
  add(b) { this.boids.push(b); }
  run() { this.boids.forEach(b => b.run(this.boids)); }
}

// — Boid —————

class Boid {
  constructor(x, y, word, brightness) {
    this.pos      = createVector(x, y);
    this.vel      = p5.Vector.random2D().mult(random(0.8, 1.6));
    this.acc      = createVector(0, 0);
    this.ms       = random(1.8, 2.6); // max speed
    this.mf       = 0.05;             // max force — weich = flüssige Kurven
    this.word     = word;
    this.brightness = brightness;
  }

  run(bs) {
    // Separation schwächer, Ausrichtung + Kohäsion stärker → enger aber flexibler Schwarm
    this.acc.add(this.separate(bs).mult(1.0));
    this.acc.add(this.align(bs).mult(1.4));
    this.acc.add(this.cohesion(bs).mult(1.3));

    this.vel.add(this.acc).limit(this.ms);
    this.pos.add(this.vel);
    this.acc.set(0, 0);

    this.borders();
    this.render();
  }

  // Nachbarn im Radius finden und fn darauf anwenden, Durchschnitt zurückgeben
  steer(bs, dist, fn) {
    let sum = createVector(0, 0), count = 0;
    for (let o of bs) {
      let d = p5.Vector.dist(this.pos, o.pos);
      if (d > 0 && d < dist) { fn(sum, o, d); count++; }
    }
    return count > 0 ? sum.div(count) : createVector(0, 0);
  }

  // Lenkkraft: gewünschte Richtung minus aktuelle Geschwindigkeit, begrenzt auf mf
  steerLimit(s) {
    return s.normalize().mult(this.ms).sub(this.vel).limit(this.mf);
  }

  // Zum Zielpunkt t steuern
  seek(t) {
    return this.steerLimit(p5.Vector.sub(t, this.pos));
  }
}

```

```

// Regel 1: Abstand halten — Abstoßung von zu nahen Nachbarn (Radius 45 px)
separate(bs) {
  let s = this.steer(bs, 45, (sum, o, d) =>
    sum.add(p5.Vector.sub(this.pos, o.pos).normalize().div(d)));
  return s.mag() > 0 ? this.steerLimit(s) : s;
}

// Regel 2: Ausrichtung — gleiche Richtung wie Nachbarn (Radius 90 px)
align(bs) {
  let s = this.steer(bs, 90, (sum, o) => sum.add(o.vel));
  return s.mag() > 0 ? this.steerLimit(s) : s;
}

// Regel 3: Kohäsion — zum Mittelpunkt der Nachbarn bewegen (Radius 160 px)
cohesion(bs) {
  let s = this.steer(bs, 160, (sum, o) => sum.add(o.pos));
  return s.mag() > 0 ? this.seek(s) : s;
}

// Rand-Teleport: links raus = rechts rein, oben raus = unten rein
borders() {
  for (let [ax, max] of [['x', width], ['y', height]]) {
    if (this.pos[ax] < -50) this.pos[ax] = max + 50;
    if (this.pos[ax] > max + 50) this.pos[ax] = -50;
  }
}

// Als Text zeichnen — Helligkeit bestimmt die Graustufe
render() {
  push();
  translate(this.pos.x, this.pos.y);
  noStroke();
  textAlign(CENTER, CENTER);
  fill(this.brightness);
  textSize(11);
  text(this.word, 0, 0);
  pop();
}
}

```

{Mittwoch} 27.05.2026

Vormittag

- Coaching-Gespräch mit Yann
- Experimente mit Flock und Typografie
 - Code von Yann verstehen

Nachmittag

- Code überarbeiten & Typografie hineinbringen
- erste Versuche Applikation audioreaktiv zu gestalten
- Austausch mit Alina

☆ FLOCK – TYPOGRAFIE GRUPPEN-FARBE

```
// noprotect

// -----
// Wörter hier eintragen – beliebig viele
// -----
const WORDS = ["D'n'B'", 'Jungle', 'Drum&Bass', 'Windrush', 'history'];

// -----
// Jedes Wort bekommt eine feste Farbe (H, S, B)
// -----
const WORD_COLORS = {};
function buildWordColors() {
  const step = 360 / WORDS.length;
  WORDS.forEach((w, i) => {
    WORD_COLORS[w] = [i * step, 70, 95]; // gleichmäßig über den Farbkreis verteilt
  });
}

// -----
// Globale Variablen & p5.js-Einstiegspunkte
// -----

let flock;

function setup() {
  createCanvas(windowWidth, windowHeight);
  colorMode(HSB, 360, 100, 100, 100);
  textFont('Lobular');
  buildWordColors();
  flock = new Flock();
  for (let i = 0; i < 100; i++)
    flock.add(new Boid(width/2, height/2));
}

function draw() {
  background(300, 100, 32); // entspricht ungefähr rgb(82,0,69) in HSB
  flock.run();
}

function mouseDragged() {
  flock.add(new Boid(mouseX, mouseY));
}
```

```

// -----
// Klasse Flock – der Schwarm
// -----

class Flock {
  constructor() {
    this.boids = [];
  }
  add(b) { this.boids.push(b); }
  run() {
    this.boids.forEach(b => b.run(this.boids));
  }
}

// -----
// Klasse Boid – ein einzelnes Tier im Schwarm
// -----

class Boid {
  constructor(x, y) {
    this.pos = createVector(x, y);
    this.vel = createVector(random(-1,1), random(-1,1));
    this.acc = createVector(0, 0);
    this.r = 1;
    this.ms = 3;
    this.mf = 0.03;
    this.word = random(WORDS);
    this.fs = random(13, 19);
    this.col = WORD_COLORS[this.word]; // feste Farbe für dieses Wort
  }

  run(bs) {
    this.acc.add(this.separate(bs).mult(1.2));
    this.acc.add(this.align(bs));
    this.acc.add(this.cohesion(bs));

    this.vel.add(this.acc).limit(this.ms);
    this.pos.add(this.vel);
    this.acc.set(0, 0);

    this.borders();
    this.render();
  }

  steer(bs, dist, fn) {
    let sum = createVector(0, 0);
    let count = 0;
    for (let o of bs) {
      let d = p5.Vector.dist(this.pos, o.pos);
      if (d > 0 && d < dist) { fn(sum, o, d); count++; }
    }
    return count > 0 ? sum.div(count) : createVector(0, 0);
  }

  limit(s) {

```

```

    return s.normalize().mult(this.ms).sub(this.vel).limit(this.mf);
  }

  seek(t) {
    return this.limit(p5.Vector.sub(t, this.pos));
  }

  // Separation: Radius von 10 → 28 erhöht damit Wörter lesbar bleiben
  separate(bs) {
    let s = this.steer(bs, 20, (sum, o, d) =>
      sum.add(p5.Vector.sub(this.pos, o.pos).normalize().div(d)));
    return s.mag() > 0 ? this.limit(s) : s;
  }

  // Align: Radius von 50 → 80, stärkere Ausrichtung = engere Schwärme
  align(bs) {
    let s = this.steer(bs, 100, (sum, o) => sum.add(o.vel));
    return s.mag() > 0 ? this.limit(s) : s;
  }

  // Cohesion: Radius von 50 → 80, Boids ziehen sich stärker zusammen
  cohesion(bs) {
    let s = this.steer(bs, 100, (sum, o) => sum.add(o.pos));
    return s.mag() > 0 ? this.seek(s) : s;
  }

  borders() {
    for (let [ax, max] of [['x', width], ['y', height]]) {
      if (this.pos[ax] < -this.r) this.pos[ax] = max + this.r;
      if (this.pos[ax] > max + this.r) this.pos[ax] = -this.r;
    }
  }

  render() {
    push();
    translate(this.pos.x, this.pos.y);
    rotate(this.vel.heading());
    textAlign(CENTER, CENTER);
    textSize(this.fs);
    fill(this.col[0], this.col[1], this.col[2]);
    noStroke();
    text(this.word, 0, 0);
    pop();
  }
}

```

```

// -----
// Globale Variablen & p5.js-Einstiegspunkte
// -----

// noprotect – sagt dem p5.js-Editor: "Bitte nicht eingreifen,
//          auch wenn eine Schleife lang läuft."
let flock; // Wird später mit einem Flock-Objekt befüllt

// setup() wird von p5.js EINMAL beim Start aufgerufen
function setup() {
  createCanvas(windowWidth, windowHeight); // Zeichenfläche = ganzes Browserfenster
  flock = new Flock();                     // Leerer Schwarm wird angelegt
  for (let i = 0; i < 200; i++)
    flock.add(new Boid(width/2, height/2)); // Anz. Boids starten in Mitte definieren
}

// draw() wird von p5.js ca. 60× pro Sekunde aufgerufen – jeder Aufruf = 1 Frame
function draw() {
  background(82, 0, 69); // Hintergrund-Farbe (löscht das vorherige Frame)
  let x = map(mouseX, 0, width, 0.00001, 6)
  let y = map(mouseY, 0, height, 0.00001, 1)
  flock.run(2.0, y, x); // Jeden Boid bewegen und zeichnen
  //noLoop()
}

// Wenn die Maus gedrückt und gezogen wird, erscheinen neue Boids
function mouseDragged() {
  flock.add(new Boid(mouseX, mouseY)); // Neuer Boid genau an der Mausposition
}

// -----
// Klasse Flock – der Schwarm
// -----

// Flock ist nur ein Behälter: eine Liste aller Boids plus zwei Hilfsmethoden.
// Der Schwarm selbst "denkt" nicht – das macht jeder Boid für sich.
class Flock {
  constructor() {
    this.boids = []; // Leeres Array, das alle Boid-Objekte hält
  }
  add(b) { this.boids.push(b); } // Einen neuen Boid zur Liste hinzufügen
  run(s, a, c) {
    // Jeden Boid in der Liste einmal ausführen.
    // Jeder bekommt die GESAMTE Liste (this.boids), damit er
    // seine Nachbarn selbst herausfiltern kann.
    this.boids.forEach(b => b.run(this.boids, s, a, c));
  }
}

// -----
// Klasse Boid – ein einzelnes Tier im Schwarm
// -----

class Boid {
  constructor(x, y) {
    // pos = aktuelle Position auf der Leinwand (x/y-Koordinaten)

```



```

    this.pos = createVector(x, y);
    // vel = Geschwindigkeit: wohin und wie schnell bewegt er sich gerade?
    // Zufällige Startwerte → jeder Boid fliegt am Anfang in eine andere Richtung
    this.vel = createVector(random(-1,1), random(-1,1));
    // acc = Beschleunigung: welche Kraft wirkt gerade auf ihn?
    // Startet immer bei (0,0) – kein Schub am Anfang
    this.acc = createVector(0, 0);
    this.r = 5;    // r = Radius/Größe des Dreiecks in Pixeln
    this.ms = 15;   // ms = max speed – wie schnell darf er maximal sein?
    this.mf = 1.833; // mf = max force – wie stark darf er seine Richtung ändern?
                    // (kleiner Wert = träges, realistisches Lenken)
}

// run() wird jeden Frame einmal aufgerufen.
// bs = das Array aller Boids (um Nachbarn zu finden)
run(bs, s, a, c) {
    // — Schritt 1: Kräfte berechnen und aufaddieren —————
    this.acc.add(this.separate(bs).mult(s)); // Abstand halten (1.5× stärker gewichtet)
    this.acc.add(this.align(bs).mult(a));    // Gleiche Richtung wie Nachbarn
    this.acc.add(this.cohesion(bs).mult(c)); // Zum Zentrum der Nachbarn bewegen

    // — Schritt 2: Physik anwenden —————
    this.vel.add(this.acc).limit(this.ms); // Geschwindigkeit += Beschleunigung, aber max. ms
    this.pos.add(this.vel);                // Position += Geschwindigkeit
    this.acc.set(0, 0);                    // Beschleunigung auf 0 zurücksetzen (wird nächsten Frame
neu berechnet)

    // — Schritt 3: Zeichnen —————
    this.borders(); // Rand-Teleport: am rechten Rand raus = links wieder rein
    this.render();  // Dreieck auf die Leinwand zeichnen
}

// — Hilfsmethode: Nachbarn im Radius finden —————
// dist = wie weit schaut der Boid?
// fn = was soll mit jedem Nachbar gemacht werden? (als Funktion übergeben)
// Gibt den Durchschnittsvektor aller gefundenen Nachbarn zurück.
steer(bs, dist, fn) {
    let sum = createVector(0, 0); // Hier werden die Vektoren aufaddiert
    let count = 0;                // Zählt, wie viele Nachbarn gefunden wurden
    for (let o of bs) {
        let d = p5.Vector.dist(this.pos, o.pos); // Abstand zu diesem Boid berechnen
        // d > 0 schließt den Boid selbst aus (Abstand zu sich selbst = 0)
        if (d > 0 && d < dist) { fn(sum, o, d); count++; }
    }
    // Durchschnitt zurückgeben, oder Nullvektor wenn keine Nachbarn gefunden
    return count > 0 ? sum.div(count) : createVector(0, 0);
}

// Normalisiert einen Vektor zu einer Lenkkraft:
// "Ich will in Richtung s, mit Maximalgeschwindigkeit, abzüglich meiner aktuellen Fahrtrichtung"
// Das Ergebnis ist die nötige Kurskorrektur.
limit(s) {
    return s.normalize().mult(this.ms).sub(this.vel).limit(this.mf);
}

// Berechnet die Lenkkraft in Richtung eines Zielpunkts t
seek(t) {
    return this.limit(p5.Vector.sub(t, this.pos)); // Richtungsvektor zum Ziel berechnen
}

```

```

}

// — REGEL 1: Separation —————
// "Halte Abstand!" – Boids die zu nah sind, stoßen sich ab.
// Radius 25 px: nur sehr nahe Nachbarn zählen.
// Je näher der Nachbar, desto stärker die Abstoßung (÷ d).
separate(bs) {
  let s = this.steer(bs, 125, (sum, o, d) =>
    sum.add(p5.Vector.sub(this.pos, o.pos).normalize().div(d)));
  // Nur lenken wenn wirklich Nachbarn da sind (mag() = Länge des Vektors)
  return s.mag() > 0 ? this.limit(s) : s;
}

// — REGEL 2: Ausrichtung —————
// "Flieg in dieselbe Richtung wie deine Nachbarn!"
// Radius 50 px: weiter schauen als bei Separation.
// Mittelt die Geschwindigkeitsvektoren (vel) aller Nachbarn.
align(bs) {
  let s = this.steer(bs, 150, (sum, o) => sum.add(o.vel));
  return s.mag() > 0 ? this.limit(s) : s;
}

// — REGEL 3: Kohäsion —————
// "Bleib beim Schwarm!" – Bewege dich zur Mitte deiner Nachbarn.
// Radius 50 px. Mittelt die Positionen (pos) aller Nachbarn,
// dann wird seek() benutzt um dorthin zu steuern.
cohesion(bs) {
  let s = this.steer(bs, 150, (sum, o) => sum.add(o.pos));
  return s.mag() > 0 ? this.seek(s) : s;
}

// Rand-Teleport: verlässt ein Boid die Leinwand, kommt er auf der anderen Seite rein.
// Die for-Schleife macht das kompakt für x und y gleichzeitig.
borders() {
  for (let [ax, max] of [['x', width], ['y', height]]) {
    if (this.pos[ax] < -this.r) this.pos[ax] = max + this.r; // Links raus → rechts rein
    if (this.pos[ax] > max + this.r) this.pos[ax] = -this.r; // Rechts raus → links rein
  }
}

// —————
// Den Boid als kleines Dreieck zeichnen, das in Flugrichtung zeigt.
// —————

render() {
  push(); // Aktuellen Zustand des Koordinatensystems
speichern
  translate(this.pos.x, this.pos.y); // Nullpunkt auf die Boid-Position verschieben
  rotate(this.vel.heading() + radians(90)); // In Flugrichtung drehen (+90° weil Dreieck nach
oben zeigt)
  fill(255, 217, 249); // Füllfarbe: helles Rosa
  stroke(252, 121, 230); // Umrandungsfarbe: kräftiges Pink
  beginShape();
  vertex( 0, -this.r * 2); // Spitze (oben/vorne)
  vertex(-this.r, this.r * 2); // Linke untere Ecke
  vertex( this.r, this.r * 2); // Rechte untere Ecke
  endShape(CLOSE);
}

```

```

    pop(); // Koordinatensystem wieder zurücksetzen (für den nächsten Boid)
  }
}

```

☆ NEUES FLOCK – TYPO-ZEICHEN ANSTELLE VON DREIECK

```

// noprotect

const PALETTE = [
  "#ffffff", "#a5b2cf", "#0015ad"
];
const wordColorMap = {};
let colorIndex = 0;
function getWordColor(word) {
  const key = word.toLowerCase();
  if (!wordColorMap[key]) {
    wordColorMap[key] = PALETTE[colorIndex++ % PALETTE.length];
  }
  return wordColorMap[key];
}

//let wordPool = ["△", "▲"]
let wordPool = ["△", "▲"]
// let wordPool = ["☆", "★", "△", "▲"]
let wordInput;
let flock;

function setup() {
  createCanvas(windowWidth, windowHeight);
  flock = new Flock();
  for (let i = 0; i < 200; i++)
    flock.add(new Boid(width/2, height/2));

  // Eingabefeld für neue Wörter
  wordInput = createInput('');
  wordInput.position(width/2 - 140, 12);
  wordInput.size(280, 30);
  wordInput.attribute('placeholder', 'Wort eingeben, Enter = hinzufügen');
  wordInput.elt.addEventListener('keydown', e => {
    if (e.key === 'Enter') {
      const words = wordInput.value().trim().split(/\s+/).filter(w => w.length > 0);
      words.forEach(w => {
        if (!wordPool.includes(w)) wordPool.push(w);
        getWordColor(w);
        flock.add(new Boid(width/2, height/2));
      });
      flock.boids.forEach(b => {

```

```

        b.word = wordPool[floor(random(wordPool.length))];
    });
    wordInput.value('');
}
});
}

function draw() {
    background(0, 30, 255, 10);
    let x = map(mouseX, 0, width, 0.00001, 6);
    let y = map(mouseY, 0, height, 0.00001, 1);
    flock.run(2.0, y, x);
}

function mouseDragged() {
    flock.add(new Boid(mouseX, mouseY));
}

// -----
// Klasse Flock - der Schwarm
// -----
class Flock {
    constructor() { this.boids = []; }
    add(b) { this.boids.push(b); }
    run(s, a, c) {
        this.boids.forEach(b => b.run(this.boids, s, a, c));
    }
}

// -----
// Klasse Boid - ein einzelnes Tier im Schwarm
// -----
class Boid {
    constructor(x, y) {
        this.pos = createVector(x, y);
        this.vel = createVector(random(-1,1), random(-1,1));
        this.acc = createVector(0, 0);
        this.r = 5;
        this.ms = 15;
        this.mf = 1.833;
        // Zufälliges Wort aus dem Pool beim Erstellen zuweisen
        this.word = wordPool[floor(random(wordPool.length))];
    }

    run(bs, s, a, c) {
        this.acc.add(this.separate(bs).mult(s));
        this.acc.add(this.align(bs).mult(a));
        this.acc.add(this.cohesion(bs).mult(c));
        this.vel.add(this.acc).limit(this.ms);
        this.pos.add(this.vel);
        this.acc.set(0, 0);
        this.borders();
        this.render();
    }

    steer(bs, dist, fn) {
        let sum = createVector(0, 0);
        let count = 0;

```

```

    for (let o of bs) {
        let d = p5.Vector.dist(this.pos, o.pos);
        if (d > 0 && d < dist) { fn(sum, o, d); count++; }
    }
    return count > 0 ? sum.div(count) : createVector(0, 0);
}

limit(s) {
    return s.normalize().mult(this.ms).sub(this.vel).limit(this.mf);
}

seek(t) {
    return this.limit(p5.Vector.sub(t, this.pos));
}

separate(bs) {
    let s = this.steer(bs, 125, (sum, o, d) =>
        sum.add(p5.Vector.sub(this.pos, o.pos).normalize().div(d)));
    return s.mag() > 0 ? this.limit(s) : s;
}

align(bs) {
    let s = this.steer(bs, 150, (sum, o) => sum.add(o.vel));
    return s.mag() > 0 ? this.limit(s) : s;
}

cohesion(bs) {
    let s = this.steer(bs, 150, (sum, o) => sum.add(o.pos));
    return s.mag() > 0 ? this.seek(s) : s;
}

borders() {
    for (let [ax, max] of [['x', width], ['y', height]]) {
        if (this.pos[ax] < -this.r) this.pos[ax] = max + this.r;
        if (this.pos[ax] > max + this.r) this.pos[ax] = -this.r;
    }
}

// -----
// Den Boid als Wort zeichnen, das in Flugrichtung zeigt.
// Gleiche Wörter → gleiche Farbe (aus wordColorMap)
// -----
render() {
    push();
    translate(this.pos.x, this.pos.y);
    rotate(this.vel.heading() + radians(90)); // In Flugrichtung drehen
    fill(getWordColor(this.word));
    noStroke();
    textAlign(CENTER, CENTER);
    textSize(this.r * 2.2); // Schriftgröße proportional zu r
    text(this.word, 0, 0);
    pop();
}
}

```

{Donnerstag} 28.05.2026

Vormittag

- Coaching-Gespräch mit Yann
- Weiterarbeit am Code

Nachmittag

- Weiterarbeit am Code
 - Input-Feld für Text im UI
 - Erste Versuche neuen Code Audioreaktiv zu machen.

★ AUDIOREACTIVE TEST-FLOCK

```
// noprotect

const PALETTE = [
  "#ffffff", "#a5b2cf", "#0015ad"
];
const wordColorMap = {};
let colorIndex = 0;
function getWordColor(word) {
  const key = word.toLowerCase();
  if (!wordColorMap[key]) {
    wordColorMap[key] = PALETTE[colorIndex++ % PALETTE.length];
  }
  return wordColorMap[key];
}

let wordPool = ["△", "▲"]
let wordInput;
let flock;

function setup() {
  createCanvas(windowWidth, windowHeight);
  setupAudio(true); // Mikrofon aktivieren
  a5.ease = .15;    // Glättung der FFT-Werte (träger = ruhiger)
  flock = new Flock();
  for (let i = 0; i < 200; i++)
```

```

    flock.add(new Boid(width/2, height/2));

    // Eingabefeld für neue Wörter
    wordInput = createInput('');
    wordInput.position(width/2 - 140, 12);
    wordInput.size(280, 30);
    wordInput.attribute('placeholder', 'Wort eingeben, Enter = hinzufügen');
    wordInput.elt.addEventListener('keydown', e => {
      if (e.key === 'Enter') {
        const words = wordInput.value().trim().split(/\s+/).filter(w => w.length > 0);
        words.forEach(w => {
          if (!wordPool.includes(w)) wordPool.push(w);
          getWordColor(w);
          flock.add(new Boid(width/2, height/2));
        });
        flock.boids.forEach(b => {
          b.word = wordPool[floor(random(wordPool.length))];
        });
        wordInput.value('');
      }
    });
  }

function draw() {
  updateAudio(); // Audio-Analyse jeden Frame aktualisieren

  // Bass (tiefe Frequenzen) aus FFT: Index 0-2 mitteln
  // --> Zahl anpassen
  let bass = (1.5*(fftEase[0] + fftEase[1] + fftEase[2]) / 3);

  // Helle Blitze bei starkem Bass (Hintergrund kurz aufhellen)
  let bgAlpha = map(bass, 0, 255, 10, 80);
  background(0, 30, 255, bgAlpha);

  let x = map(mouseX, 0, width, 0.00001, 6);
  let y = map(mouseY, 0, height, 0.00001, 1);

  // Bass steuert Separation – bei Beat weichen Boids stärker aus
  let separation = map(bass, 0, 255, 1.5, 4.0);
  flock.run(separation, y, x);
}

function mouseDragged() {
  flock.add(new Boid(mouseX, mouseY));
}

// -----
// Klasse Flock – der Schwarm
// -----
class Flock {
  constructor() { this.boids = []; }
  add(b) { this.boids.push(b); }
  run(s, a, c) {
    this.boids.forEach(b => b.run(this.boids, s, a, c));
  }
}

// -----

```

```
// Klasse Boid – ein einzelnes Tier im Schwarm
// -----
class Boid {
  constructor(x, y) {
    this.pos = createVector(x, y);
    this.vel = createVector(random(-1,1), random(-1,1));
    this.acc = createVector(0, 0);
    this.r = 5;
    this.ms = 15;
    this.mf = 1.833;
    // Zufälliges Wort aus dem Pool beim Erstellen zuweisen
    this.word = wordPool[floor(random(wordPool.length))];
    // Jedem Boid einen eigenen FFT-Index zuweisen → verschiedene Frequenzen steuern verschiedene
Boids
    this.fftIndex = floor(random(fftEase.length));
  }

  run(bs, s, a, c) {
    this.acc.add(this.separate(bs).mult(s));
    this.acc.add(this.align(bs).mult(a));
    this.acc.add(this.cohesion(bs).mult(c));
    this.vel.add(this.acc).limit(this.ms);
    this.pos.add(this.vel);
    this.acc.set(0, 0);
    this.borders();
    this.render();
  }

  steer(bs, dist, fn) {
    let sum = createVector(0, 0);
    let count = 0;
    for (let o of bs) {
      let d = p5.Vector.dist(this.pos, o.pos);
      if (d > 0 && d < dist) { fn(sum, o, d); count++; }
    }
    return count > 0 ? sum.div(count) : createVector(0, 0);
  }

  limit(s) {
    return s.normalize().mult(this.ms).sub(this.vel).limit(this.mf);
  }

  seek(t) {
    return this.limit(p5.Vector.sub(t, this.pos));
  }

  separate(bs) {
    let s = this.steer(bs, 125, (sum, o, d) =>
      sum.add(p5.Vector.sub(this.pos, o.pos).normalize().div(d)));
    return s.mag() > 0 ? this.limit(s) : s;
  }

  align(bs) {
    let s = this.steer(bs, 150, (sum, o) => sum.add(o.vel));
    return s.mag() > 0 ? this.limit(s) : s;
  }

  cohesion(bs) {

```



```

    let s = this.steer(bs, 150, (sum, o) => sum.add(o.pos));
    return s.mag() > 0 ? this.seek(s) : s;
  }

  borders() {
    for (let [ax, max] of [['x', width], ['y', height]]) {
      if (this.pos[ax] < -this.r)      this.pos[ax] = max + this.r;
      if (this.pos[ax] > max + this.r) this.pos[ax] = -this.r;
    }
  }

  // -----
  // Den Boid als Wort zeichnen, das in Flugrichtung zeigt.
  // Gleiche Wörter → gleiche Farbe (aus wordColorMap)
  // Eigene FFT-Frequenz steuert Schriftgröße
  // -----
  render() {
    // Schriftgröße pulsiert mit der eigenen Frequenz des Boids
    let freq = fftEase[this.fftIndex];
    let size = this.r * 2.2 + map(freq, 0, 255, 0, this.r * 4);

    push();
    translate(this.pos.x, this.pos.y);
    rotate(this.vel.heading() + radians(90)); // In Flugrichtung drehen
    fill(getWordColor(this.word));
    noStroke();
    textAlign(CENTER, CENTER);
    textSize(size); // Schriftgröße proportional zu r + FFT
    text(this.word, 0, 0);
    pop();
  }
}

```

★ FLOCK MIT AUDIOREAKTIVEM KREIS IN MITTE

```

// noprotect

const PALETTE = [
  "#ffffff", "#a5b2cf", "#0015ad"
];
const wordColorMap = {};
let colorIndex = 0;
function getWordColor(word) {
  const key = word.toLowerCase();
  if (!wordColorMap[key]) {
    wordColorMap[key] = PALETTE[colorIndex++ % PALETTE.length];
  }
}

```

```

    return wordColorMap[key];
}

let wordPool = ["△", "▲"]
let wordInput;
let flock;

function setup() {
  createCanvas(windowWidth, windowHeight);
  setupAudio(true); // Mikrofon aktivieren
  a5.ease = .15;    // Glättung der FFT-Werte (träger = ruhiger)
  flock = new Flock();
  for (let i = 0; i < 200; i++)
    flock.add(new Boid(width/2, height/2));

  // Eingabefeld für neue Wörter
  wordInput = createInput('');
  wordInput.position(width/2 - 140, 12);
  wordInput.size(280, 30);
  wordInput.attribute('placeholder', 'Wort eingeben, Enter = hinzufügen');
  wordInput.elt.addEventListener('keydown', e => {
    if (e.key === 'Enter') {
      const words = wordInput.value().trim().split(/\s+/).filter(w => w.length > 0);
      words.forEach(w => {
        if (!wordPool.includes(w)) wordPool.push(w);
        getWordColor(w);
        flock.add(new Boid(width/2, height/2));
      });
      flock.boids.forEach(b => {
        b.word = wordPool[floor(random(wordPool.length))];
      });
      wordInput.value('');
    }
  });
}

function draw() {
  updateAudio(); // Audio-Analyse jeden Frame aktualisieren

  // Bass (tiefe Frequenzen) aus FFT: Index 0-2 mitteln
  // ▀ --> Zahl anpassen, wenn zu leise
  let bass = (2*(fftEase[0] + fftEase[1] + fftEase[2]) / 3);

  // Helle Blitze bei starkem Bass (Hintergrund kurz aufhellen)
  let bgAlpha = map(bass, 0, 255, 10, 80);
  background(0, 30, 255);
  //   background(0, 30, 255, bgAlpha);

  // -----
  // Roter Kreis mit UK-Text im Hintergrund
  // -----

  let circleAlpha = map(bass, 0, 255, 30, 255);

  fill(220, 30, 30, circleAlpha);
  noStroke();
  ellipse(width / 2, height / 2, 300, 300);

```

```

fill(255, circleAlpha);
textAlign(CENTER, CENTER);
textSize(90);
textStyle(BOLD);
text("UK", width / 2, height / 2);
// people from english colonies / 16 / 0, 22, 89

let x = map(mouseX, 0, width, 0.00001, 6);
let y = map(mouseY, 0, height, 0.00001, 1);

// Bass steuert Separation – bei Beat weichen Boids stärker aus
//   ▀ GEÄNDERT!! -> 1.5, 4.0
let separation = map(bass, 0, 255, 0.8, 1.5);
flock.run(separation, y, x);
}

function mouseDragged() {
  flock.add(new Boid(mouseX, mouseY));
}

// -----
// Klasse Flock – der Schwarm
// -----
class Flock {
  constructor() { this.boids = []; }
  add(b) { this.boids.push(b); }
  run(s, a, c) {
    this.boids.forEach(b => b.run(this.boids, s, a, c));
  }
}

class Boid {
  constructor(x, y) {
    this.pos = createVector(x, y);
    this.vel = createVector(random(-1,1), random(-1,1));
    this.acc = createVector(0, 0);
    // ----- ▀ GEÄNDERT!! -----
    this.r = 1 //5
    this.ms = 4; // 15
    this.mf = 0.03; // 1.833
    // Zufälliges Wort aus dem Pool beim Erstellen zuweisen
    this.word = wordPool[floor(random(wordPool.length))];
    // Jedem Boid einen eigenen FFT-Index zuweisen → verschiedene Frequenzen steuern verschiedene
    Boids
    this.fftIndex = floor(random(fftEase.length));
  }
  // ----- ▀ GEÄNDERT!! -----
  run(bs, s, a, c) {
    this.acc.add(this.separate(bs).mult(1.2)); // .mult(s)
    this.acc.add(this.align(bs).mult(1)); // a
    this.acc.add(this.cohesion(bs).mult(1)); // c
    this.vel.add(this.acc).limit(this.ms);
    this.pos.add(this.vel);

    this.acc.set(0, 0);
    this.borders();
    this.render();
  }
}

```

```

steer(bs, dist, fn) {
  let sum = createVector(0, 0);
  let count = 0;
  for (let o of bs) {
    let d = p5.Vector.dist(this.pos, o.pos);
    if (d > 0 && d < dist) { fn(sum, o, d); count++; }
  }
  return count > 0 ? sum.div(count) : createVector(0, 0);
}

limit(s) {
  return s.normalize().mult(this.ms).sub(this.vel).limit(this.mf);
}

seek(t) {
  return this.limit(p5.Vector.sub(t, this.pos));
}

// -----  ▀ GEÄNDERT!!  -----
// 125
separate(bs) {
  let s = this.steer(bs, 10, (sum, o, d) =>
    sum.add(p5.Vector.sub(this.pos, o.pos).normalize().div(d)));
  return s.mag() > 0 ? this.limit(s) : s;
}
// 150
align(bs) {
  let s = this.steer(bs, 150, (sum, o) => sum.add(o.vel));
  return s.mag() > 0 ? this.limit(s) : s;
}
// 150
cohesion(bs) {
  let s = this.steer(bs, 90, (sum, o) => sum.add(o.pos));
  return s.mag() > 0 ? this.seek(s) : s;
}

borders() {
  for (let [ax, max] of [['x', width], ['y', height]]) {
    if (this.pos[ax] < -this.r) this.pos[ax] = max + this.r;
    if (this.pos[ax] > max + this.r) this.pos[ax] = -this.r;
  }
}

// -----
// Den Boid als Wort zeichnen, das in Flugrichtung zeigt.
// Gleiche Wörter → gleiche Farbe (aus wordColorMap)
// Eigene FFT-Frequenz steuert Schriftgrösse
// -----
render() {
  // Schriftgrösse pulsiert mit der eigenen Frequenz des Boids
  let freq = fftEase[this.fftIndex];
  // 2.2. zuvor
  //let size = this.r * 5 + map(freq, 0, 255, 0, this.r * 4);
  let size = 14; // fixe Schriftgrösse

  push();
  translate(this.pos.x, this.pos.y);

```

```

    rotate(this.vel.heading() + radians(90)); // In Flugrichtung drehen
    fill(getWordColor(this.word));
    noStroke();
    textAlign(CENTER, CENTER);
    textSize(size); // Schriftgrösse proportional zu r + FFT
    text(this.word, 0, 0);
    pop();
  }
}

```

★ FLOCK MIT 2 AUDIOREAKTIVEN KREISEN IN DER MITTE

```

// noprotect

const PALETTE = [
  "#ffffff", "#a5b2cf", "#0015ad"
];
const wordColorMap = {};
let colorIndex = 0;
function getWordColor(word) {
  const key = word.toLowerCase();
  if (!wordColorMap[key]) {
    wordColorMap[key] = PALETTE[colorIndex++ % PALETTE.length];
  }
  return wordColorMap[key];
}

let wordPool = ["△", "▲"]
let wordInput;
let flock;

function setup() {
  createCanvas(windowWidth, windowHeight);
  setupAudio(true); // Mikrofon aktivieren
  a5.ease = .15; // Glättung der FFT-Werte (träger = ruhiger)
  flock = new Flock();
  for (let i = 0; i < 200; i++)
    flock.add(new Boid(width/2, height/2));

  // Eingabefeld für neue Wörter
  wordInput = createInput('');
  wordInput.position(width/2 - 140, 12);
  wordInput.size(280, 30);
  wordInput.attribute('placeholder', 'Wort eingeben, Enter = hinzufügen');
  wordInput.elt.addEventListener('keydown', e => {
    if (e.key === 'Enter') {
      const words = wordInput.value().trim().split(/\s+/).filter(w => w.length > 0);
      words.forEach(w => {
        if (!wordPool.includes(w)) wordPool.push(w);
      });
    }
  });
}

```

```

        getWordColor(w);
        flock.add(new Boid(width/2, height/2));
    });
    flock.boids.forEach(b => {
        b.word = wordPool[floor(random(wordPool.length))];
    });
    wordInput.value('');
}
});
}

// -----
// Audioreaktivität
// -----

function draw() {
    updateAudio(); // Audio-Analyse jeden Frame aktualisieren

    // Bass (tiefe Frequenzen) aus FFT: Index 0-2 mitteln
    // ▮ --> Zahl anpassen, wenn zu leise
    let bass = (1.5*(fftEase[0] + fftEase[1] + fftEase[2]) / 3);
    let mid = (fftEase[5] + fftEase[6] + fftEase[7]) / 3;

    // Helle Blitze bei starkem Bass (Hintergrund kurz aufhellen)
    let bgAlpha = map(bass, 0, 255, 10, 80);
    background(0, 30, 255);
    // background(0, 30, 255, bgAlpha);

    // — Zwei audioreaktive Kreise, links und rechts ohne Überschneidung —
    let alpha1 = map(bass, 0, 255, 30, 255); // Transparenz via Bass
    let alpha2 = map(mid, 0, 255, 30, 255); // Transparenz via Mitten
    let high = (fftEase[10] + fftEase[11] + fftEase[12]) / 3;
    let circleSize = map(high, 0, 255, 380, 520); // Grösse pulsiert

    let offset = width / 4; // dynamisch — passt sich an Fenstergrösse an

    // -----
    // Kreise
    // -----

    // Linker Kreis — reagiert auf Mitten
    fill(0, 245, 118, alpha2);
    noStroke();
    ellipse(width / 2 - offset, height / 2, 300, 300);

    // Text linker Kreis: Caribbean
    fill(255, alpha1);
    textAlign(CENTER, CENTER);
    textSize(40);
    textStyle(BOLD);
    text("CARRIBEAN", width / 2 - offset, height / 2);

    // Rechter Kreis — reagiert auf Bass
    fill(220, 30, 30, alpha1);
    noStroke();
    ellipse(width / 2 + offset, height / 2, 300, 300);

    // Text rechter Kreis: UK

```

```

fill(255, alpha2);
textAlign(CENTER, CENTER);
textSize(80);
textStyle(BOLD);
text("UK", width / 2 + offset, height / 2);

//// Mittlerer Kreis – reagiert auf Höhen mit Pulsieren & Transparenz
// fill(255, 255, 255, alpha1);
// noStroke();
// ellipse(width / 2, height / 2, circleSize, circleSize);

// fill(255);
// textAlign(CENTER, CENTER);
// textSize(20);
// textStyle(BOLD);
// text("people from english colonies", width / 2, height / 2);

// -----

let x = map(mouseX, 0, width, 0.00001, 6);
let y = map(mouseY, 0, height, 0.00001, 1);

// Bass steuert Separation – bei Beat weichen Boids stärker aus
//   ▀ GEÄNDERT!! -> 1.5, 4.0
let separation = map(bass, 0, 255, 0.8, 1.5);
flock.run(separation, y, x);
}

function mouseDragged() {
  flock.add(new Boid(mouseX, mouseY));
}

// -----
// Klasse Flock – der Schwarm
// -----
class Flock {
  constructor() { this.boids = []; }
  add(b) { this.boids.push(b); }
  run(s, a, c) {
    this.boids.forEach(b => b.run(this.boids, s, a, c));
  }
}

// -----
// Klasse Boid – ein einzelnes Tier im Schwarm
// -----
class Boid {
  constructor(x, y) {
    this.pos = createVector(x, y);
    this.vel = createVector(random(-1,1), random(-1,1));
    this.acc = createVector(0, 0);
    // ----- ▀ GEÄNDERT!! -----
    this.r = 1 //5
    this.ms = 4; // 15
    this.mf = 0.03; // 1.833
    // Zufälliges Wort aus dem Pool beim Erstellen zuweisen
    this.word = wordPool[floor(random(wordPool.length))];
  }
}

```

```

    // Jedem Boid einen eigenen FFT-Index zuweisen → verschiedene Frequenzen steuern verschiedene
Boids
    this.fftIndex = floor(random(fftEase.length));
}
// ----- ► GEÄNDERT!! -----
run(bs, s, a, c) {
    this.acc.add(this.separate(bs).mult(1.2)); // .mult(s)
    this.acc.add(this.align(bs).mult(1)); // a
    this.acc.add(this.cohesion(bs).mult(1)); // c
    this.vel.add(this.acc).limit(this.ms);
    this.pos.add(this.vel);

    this.acc.set(0, 0);
    this.borders();
    this.render();
}

steer(bs, dist, fn) {
    let sum = createVector(0, 0);
    let count = 0;
    for (let o of bs) {
        let d = p5.Vector.dist(this.pos, o.pos);
        if (d > 0 && d < dist) { fn(sum, o, d); count++; }
    }
    return count > 0 ? sum.div(count) : createVector(0, 0);
}

limit(s) {
    return s.normalize().mult(this.ms).sub(this.vel).limit(this.mf);
}

seek(t) {
    return this.limit(p5.Vector.sub(t, this.pos));
}

// ----- ► GEÄNDERT!! -----
// 125
separate(bs) {
    let s = this.steer(bs, 10, (sum, o, d) =>
        sum.add(p5.Vector.sub(this.pos, o.pos).normalize().div(d)));
    return s.mag() > 0 ? this.limit(s) : s;
}
// 150
align(bs) {
    let s = this.steer(bs, 150, (sum, o) => sum.add(o.vel));
    return s.mag() > 0 ? this.limit(s) : s;
}
// 150
cohesion(bs) {
    let s = this.steer(bs, 50, (sum, o) => sum.add(o.pos));
    return s.mag() > 0 ? this.seek(s) : s;
}

borders() {
    for (let [ax, max] of [['x', width], ['y', height]]) {
        if (this.pos[ax] < -this.r) this.pos[ax] = max + this.r;
        if (this.pos[ax] > max + this.r) this.pos[ax] = -this.r;
    }
}

```



```
}

// -----
// Den Boid als Wort zeichnen, das in Flugrichtung zeigt.
// Gleiche Wörter → gleiche Farbe (aus wordColorMap)
// Eigene FFT-Frequenz steuert Schriftgröße
// -----
render() {
  // Schriftgröße pulsiert mit der eigenen Frequenz des Boids
  let freq = fftEase[this.fftIndex];
  // 2.2. zuvor
  //let size = this.r * 5 + map(freq, 0, 255, 0, this.r * 4);
  let size = 14; // fixe Schriftgröße

  push();
  translate(this.pos.x, this.pos.y);
  rotate(this.vel.heading() + radians(90)); // In Flugrichtung drehen
  fill(getWordColor(this.word));
  noStroke();
  textAlign(CENTER, CENTER);
  textSize(size); // Schriftgröße proportional zu r + FFT
  text(this.word, 0, 0);
  pop();
}
}
```

{Freitag} 29.05.2026

Vormittag

- Absprache mit Alina, gegenseitiger Support
- Überarbeitung des Konzepts
- Vorbereitung & Abstimmung für die Präsentation

Nachmittag

- Anpassung des Typo-Codes von Alina
- letzte finale Anpassungen am Code für Präsentation
 - nicht jeder Zwischenschritt zur finalen Version ist auf den Screenshots ersichtlich
- → die finale Version ist in einem separaten Dokument ersichtlich

☆ INTRO – TYPO

```
// {"P5LIVE":{"name":"first_scene_001","mod":1780240676090}}

let meAskyou = ["The history of drum and bass and the windrush generation.",
"Did you hear about the windrush generation before?",
"What does it have to do with drum and bass?",
"And how was drum and bass being formed in the UK? "]
let questions = meAskyou.join(' ')

function setup() {
  createCanvas(windowWidth, windowHeight);
}

function draw() {
  // live wechselt zwischen 0 und 7 – steuert das textLeading (Zeilenabstand)
  let live = (frameCount%6)
  // frameRate: 2 fps → langsamer Wechsel, gut für Lesbarkeit
  frameRate(2);

  background(0, 0, 255, 150);
  fill(255)
  textSize(60)
  textWrap(WORD)
  textFont("Satoshi")
  textLeading(32*live);
  textStyle(random([ITALIC,NORMAL]))
  text(questions.repeat(random(16)),100,10,windowWidth/1.1,windowHeight);
}
```

The history of drum and bass and the windrush
generation. Did you hear about the windrush
generation. Did you hear about the windrush
generation before? What does it have to do with
drum and bass? And how was drum and bass being
formed in the UK? The history of drum and
windrush generation before? What does it have to
do with drum and bass? And how was drum and
bass being formed in the UK?

☆ FLOCK – WÖRTER

```
// noprotect

// -----
// Flocking
// Simuliert Schwarmverhalten nach Craig Reynolds (1986):
// jeder Boid folgt drei einfachen Regeln:
//   Separation – Abstand zu Nachbarn halten
//   Alignment  – Richtung der Nachbarn angleichen
//   Cohesion   – Zur Mitte der Gruppe bewegen
// -----

// Farbpalette für die Wörter – jedes Wort bekommt eine feste Farbe
const PALETTE = [
  "#ffffff", // reines Weiss
  "#ff6600", // Jungle-Orange
  "#ffcc00", // Goldgelb – Metalheadz
  "#00ff99", // Acid-Grün
  "#ff0066", // Neon-Pink
  "#ffe44d", // helles Amber – statt Lila
  "#ff3300", // Blutrot
  "#f0f000", // Neon-Gelb – statt Elektrisch-Blau
];

// Wörterbuch: speichert welches Wort welche Farbe hat
// damit gleiche Wörter immer dieselbe Farbe bekommen
const wordColorMap = {};
let colorIndex = 0;

// Gibt die Farbe eines Wortes zurück / oder Neue
function getWordColor(word) {
  const key = word.toLowerCase();
  if (!wordColorMap[key]) {
    wordColorMap[key] = PALETTE[colorIndex++ % PALETTE.length];
  }
  return wordColorMap[key];
}

// Pool der möglichen Wörter/Symbole die ein Boid anzeigen kann
//let wordPool = ["△", "▲"]
let wordPool = ["△", "▲", "Reggae", "Dub", "Hip-Hop", "Electro Funk", "Soul", "Dancehall", "Grime",
  "Punk", "Jungle", "Breakbeat"]
let wordInput;
let flock;

// setup() für den Flocking-Modus inkl. Audio
function setup() {
  createCanvas(windowWidth, windowHeight);
  setupAudio(true);
  a5.ease = .15;

  // Schwarm erstellen und 50 Boids in der Bildschirmmitte starten
  flock = new Flock();
  // ----- ► GEÄNDERT!! -----
  for (let i = 0; i < 100; i++)
    flock.add(new Boid(width/2, height/2));
}
```

```

//Eingabefeld für neue Wörter
wordInput = createInput('');
wordInput.position(width/2 - 140, 12);
wordInput.size(280, 30);
wordInput.attribute('placeholder', 'Wort eingeben, Enter = hinzufügen');
wordInput.elt.addEventListener('keydown', e => {
  if (e.key === 'Enter') {
    const words = wordInput.value().trim().split(/\s+/).filter(w => w.length > 0);
    words.forEach(w => {
      if (!wordPool.includes(w)) wordPool.push(w);
      getWordColor(w);
      flock.add(new Boid(width/2, height/2));
    });
    flock.boids.forEach(b => {
      b.word = wordPool[floor(random(wordPool.length))];
    });
    wordInput.value('');
  }
});
}

// -----
// Audioreaktivität
// -----

function draw() {
  // FFT-Analyse aktualisieren – muss jeden Frame aufgerufen werden
  updateAudio();

  // Bass: Durchschnitt der tiefsten FFT-Bins (Index 0–2), mit Faktor 2 verstärkt
  // ▀ --> Faktor anpassen wenn Bass zu schwach reagiert
  let bass = (1*(fftEase[0] + fftEase[1] + fftEase[2]) / 3);

  // ----- ▀ GEÄNDERT!! -----
  // bgAlpha wäre für transparenten Hintergrund
  let bgAlpha = map(bass, 0, 255, 10, 80);
  background(0, 30, 255);
  // background(0, 20, 255, 10)

  // Transparenz und Kreisgrösse reagieren auf Bass und Höhen
  let alpha1 = map(bass, 0, 255, 30, 255);
  let high = (fftEase[10] + fftEase[11] + fftEase[12]) / 3; // Höhen: Index 10–12
  let circleSize = map(high, 0, 255, 380, 520); // Kreisgrösse pulsiert mit Höhen

  // -----
  // Kreise im Hintergrund
  // -----

  // // 1. Blauer Kreis – reagiert auf Bass
  // let circleAlpha = map(bass, 0, 255, 30, 255);

  // fill(0, 16, 120, circleAlpha);
  // noStroke();
  // ellipse(width / 2, height / 2, 300, 300);

  // fill(255, circleAlpha);
  // textAlign(CENTER, CENTER);

```

```

// textWrap(WORD)
// textSize(20);
// textStyle(BOLD);
// text("commonwealth countrys", width / 2, height / 2);
// //commonwealth countrys

// 2. Mittlerer Kreis – reagiert auf Höhen mit Pulsieren & Transparenz
// fill(255, 255, 255, alphas);
// noStroke();
// ellipse(width / 2, height / 2, circleSize, circleSize);

fill(255);
textAlign(CENTER, CENTER);
textSize(80);
textStyle(BOLD);
text("the history of drum & bass", width / 2, height / 2);
// new sound-sytem

// -----

// Mausposition steuert Alignment (y-Achse) und Cohesion (x-Achse) des Schwarms
// map() übersetzt Pixel-Pos in einen kleinen Wertebereich (fast 0 bis max)
let x = map(mouseX, 0, width, 0.00001, 6); // Cohesion-Stärke
let y = map(mouseY, 0, height, 0.00001, 1); // Alignment-Stärke

// Bass steuert Separation – bei starkem Beat weichen Boids stärker auseinander
// ----- ■ GEÄNDERT!! -----
// ----- 1.5 --> 4.0 (höherer Wert = mehr Ausweichen)
let separation = map(bass, 0, 255, 0.8, 4.0);
flock.run(separation, y, x);
}

// Neuen Boid an der Mausposition hinzufügen beim Ziehen
function mouseDragged() {
  flock.add(new Boid(mouseX, mouseY));
}

// -----
// Klasse Flock – der Schwarm
// Verwaltet alle Boids und lässt sie jeden Frame updaten
// -----
class Flock {
  constructor() { this.boids = []; }
  add(b) { this.boids.push(b); }
  // s = separation, a = alignment, c = cohesion
  run(s, a, c) {
    this.boids.forEach(b => b.run(this.boids, s, a, c));
  }
}

// -----
// Klasse Boid – ein einzelnes Tier im Schwarm
// -----
class Boid {
  constructor(x, y) {
    this.pos = createVector(x, y); // Position
    this.vel = createVector(random(-1,1), random(-1,1)); // Startgeschwindigkeit zufällig
  }
}

```

```

    this.acc = createVector(0, 0); // Beschleunigung (wird jeden Frame neu berechnet)

    // ----- ► GEÄNDERT!! -----
    // ----- Boid Properties
    this.r = 5; // Radius: 1 --> 5
    this.ms = 15; // max speed: 4 --> 15
    this.mf = 1.833; // max force: 0.03 --> 1.833
    // Zufälliges Wort aus dem Pool beim Erstellen zuweisen
    this.word = wordPool[floor(random(wordPool.length))];
    // Jedem Boid einen eigenen FFT-Index zuweisen → verschiedene Frequenzen steuern verschiedene
Boids
    this.fftIndex = floor(random(fftEase.length));
}

// Wird jeden Frame aufgerufen: Kräfte berechnen, Position updaten, zeichnen
// ----- ► GEÄNDERT!! -----
// ----- Direction of Motion
run(bs, s, a, c) {
    this.acc.add(this.separate(bs).mult(s)); // Separation-Kraft: 1.2 --> .mult(s)
    this.acc.add(this.align(bs).mult(a)); // Alignment-Kraft: 1 --> a
    this.acc.add(this.cohesion(bs).mult(c)); // Cohesion-Kraft: 1 --> c
    this.vel.add(this.acc).limit(this.ms); // Geschwindigkeit begrenzen
    this.pos.add(this.vel); // Position updaten

    this.acc.set(0, 0); // Beschleunigung zurücksetzen für nächsten Frame
    this.borders(); // Bildschirmränder prüfen
    this.render(); // Zeichnen
}

// Hilfsfunktion: iteriert über alle Boids im Radius dist
// und wendet fn() auf jeden Nachbarn an – gibt Durchschnitt zurück
steer(bs, dist, fn) {
    let sum = createVector(0, 0);
    let count = 0;
    for (let o of bs) {
        let d = p5.Vector.dist(this.pos, o.pos);
        if (d > 0 && d < dist) { fn(sum, o, d); count++; }
    }
    return count > 0 ? sum.div(count) : createVector(0, 0);
}

// Berechnet Lenkkraft: Wunschgeschwindigkeit minus aktuelle Geschwindigkeit, begrenzt auf mf
limit(s) {
    return s.normalize().mult(this.ms).sub(this.vel).limit(this.mf);
}

// Lenkt den Boid in Richtung eines Zielpunkts t
seek(t) {
    return this.limit(p5.Vector.sub(t, this.pos));
}

// ----- ► GEÄNDERT!! -----
// SEPARATION: Maintain distance from neighbors
// 20 --> 125 (kompakt --> verteilt)
separate(bs) {
    let s = this.steer(bs, 125, (sum, o, d) =>
        sum.add(p5.Vector.sub(this.pos, o.pos).normalize().div(d))); // Je näher, desto stärker
    return s.mag() > 0 ? this.limit(s) : s;
}

```

```

}

// ALIGNMENT: Align with the direction of neighbors
// 50 --> 150 (nur nahe Nachbarn --> gross = gleichmässig)
align(bs) {
  let s = this.steer(bs, 150, (sum, o) => sum.add(o.vel));
  return s.mag() > 0 ? this.limit(s) : s;
}

// COHESION: Move toward the center of the group
// 100 --> 150 (nahe Nachbarn --> hält stark zusammen)
cohesion(bs) {
  let s = this.steer(bs, 150, (sum, o) => sum.add(o.pos));
  return s.mag() > 0 ? this.seek(s) : s;
}

// Teleportiert den Boid auf die gegenüberliegende Seite wenn er den Rand überschreitet
borders() {
  for (let [ax, max] of [['x', width], ['y', height]]) {
    if (this.pos[ax] < -this.r)      this.pos[ax] = max + this.r;
    if (this.pos[ax] > max + this.r) this.pos[ax] = -this.r;
  }
}

// -----
// Den Boid als Wort zeichnen, das in Flugrichtung zeigt.
// Gleiche Wörter → gleiche Farbe (aus wordColorMap)
// -----
render() {
  let size = 14; // fixe Schriftgrösse

  //Zum Aktivieren: Kommentarzeichen entfernen
  push();
  translate(this.pos.x, this.pos.y);
  rotate(this.vel.heading() + radians(90)); // In Flugrichtung drehen
  fill(getWordColor(this.word));
  noStroke();
  textAlign(CENTER, CENTER);
  textSize(size); // Schriftgrösse proportional zu r + FFT
  text(this.word, 0, 0);
  pop();
}
}

```